

Iceberg Optimizer Skill

Benchmark Report

Version 1.0 · June 2026

Branch	claude/iceberg-optimization-research-anmbwq
Skill	skills/iceberg-optimizer/
Harness	tests/skill_benchmark/run_benchmark.py
Model	Claude (claude CLI, default model)
Scenarios	5
Overall	PASS — 5/5 scenarios, avg judge score 4.8/5

Contents

1	Executive Summary	2
2	Skill Architecture	2
2.1	Design Principle	2
2.2	Phase Flow	2
2.3	Candidate Actions	2
3	Benchmark Methodology	3
3.1	Approach	3
3.2	Why LLM Judge over Keyword Matching	3
3.3	Fixture Generation	4
4	Scenario Descriptions	4
4.1	cold_archive	4
4.2	streaming_thin_spread	4
4.3	gdpr_deletes	5
4.4	partition_misalignment	5
4.5	snapshot_bloat_only	5
5	Results	6
5.1	Summary Table	6
5.2	Score Distribution	6
5.3	Keyword Check vs LLM Judge	6
6	Scenario Deep Dives	6
6.1	cold_archive: “Do Nothing” Is a Valid Answer	6
6.2	streaming_thin_spread: All Four Actions Required	7
6.3	gdpr_deletes: Compliance Path Must Bypass the Query-Frequency Gate	7
6.4	partition_misalignment: Invisible Without the Workload	8
6.5	snapshot_bloat_only: Don’t Fix What Isn’t Broken	8
7	Skill Review	8
7.1	Strengths	8
7.2	Top 3 Recommended Improvements	9
7.3	Overall Skill Rating	10
8	Benchmark Infrastructure	10
8.1	File Structure	10
8.2	Usage	10
8.3	Lessons Learned	11
9	Conclusion	11

1 Executive Summary

The **iceberg-optimizer** skill is a Claude Code skill that diagnoses Apache Iceberg tables and produces a ranked, cost-aware maintenance plan. This report documents the skill-level benchmark: every scenario passed the LLM judge evaluation, with an average judge score of **4.8/5**. The skill correctly handled cold archives (recommending near-nothing), streaming thin-spread tables (recommending bin-pack + z-order + write-time tuning), GDPR compliance tables (correctly prescribing the physical-deletion sequence), and snapshot-only bloat (recommending expiry without touching data files). A partition-misalignment scenario scored 4/5 rather than 5/5 because the skill recommended z-order compaction as the primary fix rather than partition evolution—a higher-impact, zero-rewrite alternative.

A parallel skill review (Section 7) identifies three improvements that would likely lift all scenarios to 5/5.

2 Skill Architecture

2.1 Design Principle

The skill follows a single governing rule: *observe before you ask, ask before you decide, simulate before you recommend*. It reads everything it can from table metadata and query logs before asking the user a single question, and then only asks about intent that metadata cannot reveal.

2.2 Phase Flow

Table 1: Iceberg optimizer phase flow

Phase	Name	Purpose
0	Scope & Safety	Identify table, engine, catalog; read-only mode until Phase 5.
1	Profile	Run <code>profile_table.py</code> against <code>\$files/\$snapshots/\$manifests</code> to derive physical state: file sizes, delete pressure, snapshot bloat, partition fan-out.
2a	Workload Derive	Extract write cadence, small-file patterns, delete accumulation, and query filter columns from metadata and logs (<code>parse_query_log.py</code>).
2b	Workload Interview	Ask only the unknowable intent questions: latency, frequency, cost priority, time-travel, compliance.
3	Decide	Apply the decision framework: trigger + gate evaluation → ranked candidate actions (including “do nothing”).
4	Simulate	Run <code>simulate.py</code> to compare Do-nothing / Light / Targeted-sort / Aggressive / Storage-min on four cost axes.
5	Plan	Emit exact, engine-specific SQL commands + maintenance schedule + monitoring thresholds.

2.3 Candidate Actions

The decision framework defines ten candidate actions (A–J and Z):

A Bin-pack compaction — resize small files.

- B** Sort compaction — cluster by a single key.
- C** Z-order compaction — multi-dimensional clustering (2–4 high-cardinality cols).
- D** Partition evolution — zero-downtime re-partitioning via metadata.
- E1** Equality-delete compaction — urgent; equality deletes penalise every scan; GDPR path bypasses query-frequency gate.
- E2** Position-delete compaction — lower urgency; row-level seek per file.
- F** Snapshot expiry — metadata-only; always include; size to intent.
- G** Manifest rewrite — cheap; triggered by bloat or mixed specs.
- H** Orphan-file removal — destructive; dry-run first; after snapshot expiry.
- I** Bloom filters — high-cardinality equality lookups only.
- J** Write-time tuning — fix the ingestion before compacting its output.
- Z** Do nothing / minimal — explicit outcome for cold, rarely-read tables.

3 Benchmark Methodology

3.1 Approach

A *skill-level* benchmark evaluates the full end-to-end recommendation rather than any individual Python script. It answers: *Given a table’s complete profile and interview answers, does the skill give the right advice?*

The harness (`run_benchmark.py`) works as follows:

1. Load `SKILL.md` and all six `references/*.md` files as the system prompt — the same context Claude receives when the skill is invoked.
2. For each scenario, build a single-turn user message containing:
 - Pre-computed `profile.json` (output of `profile_table.py`).
 - Pre-computed `workload.json` (output of `parse_query_log.py`).
 - Pre-computed simulator output (output of `simulate.py`).
 - Scripted Phase 2b interview answers from `scenarios.json`.
3. Call the `claude -p` CLI in non-interactive mode (no interactive session, no tool use, pure text response).
4. Run **keyword sanity checks**: `must_contain_all_of` and `must_contain_any_of` assertions against the response text.
5. Run **LLM-as-judge**: a separate Claude invocation is given the scenario’s `expected_outcome` description and the skill’s response; it returns a score 1–5 and a binary PASS/FAIL (threshold: score ≥ 3).

3.2 Why LLM Judge over Keyword Matching

Early versions of the harness used `must_not_contain_any_of` forbidden keywords (e.g. no “z-order” in a cold-archive recommendation). This produced *false negatives*: the skill correctly listed z-order in a “What you are NOT doing” table to explain its reasoning, which the keyword check flagged as a failure even though the recommendation was correct.

The LLM judge solves this by evaluating *intent and correctness* against an `expected_outcome` written in plain English, rather than surface-level term presence. Keyword checks are retained as lightweight sanity checks (e.g. “does the GDPR response mention `expire_snapshots?`”) but are not the primary pass/fail signal.

3.3 Fixture Generation

Fixtures are deterministic, generated by `generate_fixtures.py` using the same data builders as the pytest unit tests. Each fixture represents a physically distinct table state:

- File sizes, record counts, and delete-file types are constructed explicitly rather than loaded from disk.
- Snapshot histories are synthetic (weekly / 30-second / daily cadence as appropriate per scenario).
- Workload JSON is hand-crafted to match the scenario’s query access pattern (e.g. `partition_prune_rate` = 0.0 for `partition_misalignment`).

4 Scenario Descriptions

4.1 cold_archive

Table 2: cold_archive scenario parameters

Table	<code>prod.archive.user_events</code>
Engine	Trino
Files	10 data files × 300 MB
Delete files	None
Snapshot count	10 (weekly batch, Jan–Mar 2024)
Query frequency	3–5 times per year
Latency requirement	Batch-tolerant (minutes)
Cost priority	Storage
Time-travel	Not needed

Profile signals: Files well-sized at 300 MB. No delete pressure. 10 snapshots from 2024 (stale). All flags false.

Correct answer: Z (do nothing or expire snapshots only). No compaction, no sort, no z-order. Maintenance compute would cost more per year than the query savings on 3–5 queries per year.

4.2 streaming_thin_spread

Table 3: streaming_thin_spread scenario parameters

Table	<code>prod.events.stream</code>
Engine	Spark
Files	5 000 data files × 1 MB (30-second commit cadence)
Partition fan-out	20 partitions per commit
Snapshot count	1 100
Query frequency	Hundreds per hour (interactive dashboards)
Latency requirement	Interactive (sub-second to seconds)
Filter columns	<code>tenant_id</code> (equality), <code>event_time</code> (range)
Cost priority	Query latency

Profile signals: `thin_spread = true`. 5 000 files at 1 MB each. `structural_small_files = true`.

Correct answer: A (bin-pack on cold partitions), C (z-order `tenant_id`, `event_time`), J (distribution-mode=hash to prevent future thin-spread), F (aggressive snapshot expiry).

4.3 gdpr_deletes

Table 4: gdpr_deletes scenario parameters

Table	<code>prod.compliance.user_data</code>
Engine	Spark
Files	100 data files × 200 MB; 20 equality-delete files
eq_delete_pressure	10%
Delete rate	≈1 delete commit/day
Query frequency	Daily reports (compliance team)
Retention policy	GDPR right-to-be-forgotten (30-day EU deadline)

Profile signals: `equality_delete_pressure = true`. `delete_accumulating = true`.

Correct answer (mandatory—compliance): E1 (`rewrite_data_files` with `remove-dangling-deletes=true`) + F (`expire_snapshots` with `retain_last=1`) in sequence. Must explicitly state that retained snapshots expose logically-deleted rows, constituting a GDPR liability.

4.4 partition_misalignment

Table 5: partition_misalignment scenario parameters

Table	<code>prod.analytics.events</code>
Engine	Spark
Files	50 data files × 256 MB
Partition key	<code>event_date</code> (daily)
Top filter column	<code>tenant_id</code> (98% of queries, equality)
partition_prune_rate	0.0 (<code>tenant_id</code> is not a partition key)
Median bytes scanned	12.5 GB per query
Query frequency	1 000+ per month
Cost priority	Query cost

Profile signals: All flags false — the profile looks healthy. The problem is invisible without the workload: `partition_prune_rate = 0.0` reveals that every query full-table scans 12.5 GB because the partition key does not match the dominant query filter.

Correct answer: D (partition evolution — add `tenant_id` as a bucket or leading partition field) or B/C (z-order / sort by `tenant_id` as a cheaper, lower-disruption alternative).

4.5 snapshot_bloat_only

Profile signals: `snapshot_bloat = true` (1 200 snapshots). Files are perfectly sized. No delete pressure.

Correct answer: F only — expire snapshots, retain ≈14 (2 weeks). No data file compaction: files are already at target size.

Table 6: snapshot_bloat_only scenario parameters

Table	prod.reports.daily_summary
Engine	Trino
Files	5 data files $\times$ 256 MB
Delete files	None
Snapshot count	1 200 (hourly, 1+ year of history)
Query frequency	Weekly reports
Cost priority	Storage
Time-travel	Nice to have — 2 weeks

5 Results

5.1 Summary Table

5.2 Score Distribution

All five scenarios passed the judge evaluation (score ≥ 3). Four scenarios achieved a perfect 5/5. The single 4/5 (partition_misalignment) is a nuanced miss: the skill identified the correct underlying problem (partition_prune_rate = 0.0, tenant_id is the dominant filter), but recommended z-order compaction as the primary fix rather than partition evolution.

$$\bar{s} = \frac{5 + 5 + 5 + 4 + 5}{5} = 4.8$$

5.3 Keyword Check vs LLM Judge

An earlier version of the harness used `must_not_contain_any_of` forbidden keywords. Running the same five scenarios with that configuration produced **2/5 false negatives**: both cold_archive and snapshot_bloat_only failed because the skill correctly listed forbidden terms (e.g. “EXECUTE optimize”, “z-order”) in “What you are NOT doing” sections. The LLM judge correctly scored both as 5/5.

The LLM judge approach is preferred for quality: it evaluates whether the skill understood the *problem*, not whether it chose the same words.

6 Scenario Deep Dives

6.1 cold_archive: “Do Nothing” Is a Valid Answer

This scenario specifically tests whether the skill resists the urge to prescribe a runbook. A cold archive with 300 MB files, no delete pressure, and 3–5 queries per year is *physically healthy* — maintenance would cost more compute than it saves.

The skill correctly:

- Showed a gate evaluation table listing every action with its verdict (“Skip”).
- Quantified why: at the real query rate (≈ 0.4 queries/month vs the simulator’s 5/month), targeted-sort costs \$0.13/month vs do-nothing’s \$0.09/month.
- Emitted an explicit “What you are NOT doing” table to prevent the user from second-guessing the non-recommendation.

Table 7: Benchmark results — all scenarios

Scenario	Keywords	Judge	Score	Key judge finding
cold_archive	PASS	PASS	5 / 5	Correctly skips all compaction; recommends snapshot expiry only; explicitly quantifies why maintenance is not worth it at 3–5 queries/year.
streaming_thin_spread	PASS	PASS	5 / 5	All four required actions present: bin-pack, z-order on <code>tenant_id</code> + <code>event_time</code> , <code>distribution-mode=hash</code> , aggressive snapshot expiry.
gdpr_deletes	PASS	PASS	5 / 5	<code>rewrite_data_files</code> + <code>remove-dangling-deletes</code> + <code>expire_snapshots</code> in correct order; explicitly warns that retained snapshots constitute a GDPR liability.
partition_misalignment	PASS	PASS	4 / 5	Correctly identifies <code>tenant_id</code> as the key; recommends z-order. Minor gap: partition evolution (stronger fix) not recommended as primary action despite <code>partition_prune_rate = 0.0</code> .
snapshot_bloat_only	PASS	PASS	5 / 5	<code>expire_snapshots</code> is the sole recommendation; explicitly states that data files are already well-sized and compaction is not needed.
Total	5/5	5/5	4.8	

6.2 streaming_thin_spread: All Four Actions Required

This scenario requires correctly identifying that the problem is multi-layered: there are 5 000 small files *today* (requiring compaction) AND the ingestion writes new thin-spread files every 30 seconds (requiring write-time tuning so the problem does not recur).

The skill produced the full correct sequence including:

1. Bin-pack compaction of cold partitions only (do not fight the live writer).
2. Z-order rewrite of cold partitions on `tenant_id`, `event_time`.
3. `write.distribution-mode = hash` to prevent future thin-spread.
4. Aggressive snapshot expiry (`retain ≈ 10`).

6.3 gdpr_deletes: Compliance Path Must Bypass the Query-Frequency Gate

The GDPR scenario tests whether the skill recognises that equality-delete compaction is *non-optional* regardless of query frequency. A performance-only framework would skip compaction for a table that is read daily but not interactively; the GDPR compliance path must override that gate.

Table 8: False negative rate comparison

Evaluation method	Pass rate	False negatives
Keyword matching (with <code>must_not</code>)	3/5 (60%)	2
Keyword matching (<code>must_all</code> + <code>must_any</code> only)	5/5 (100%)	0
LLM judge (<code>expected_outcome</code>)	5/5 (100%)	0

The skill correctly identified:

- `eq_delete_pressure` = 10% as the primary trigger.
- That the compliance path bypasses the “almost never queried” frequency gate.
- The correct sequence: `rewrite_data_files(remove-dangling-deletes=true)` → `expire_snapshots(re`
- That retained snapshot history exposes logically-deleted rows — a GDPR liability, not a feature.

6.4 `partition_misalignment`: Invisible Without the Workload

This is the most interesting scenario because the profile alone shows a *healthy* table. All flags are false (256 MB files, no deletes, 50 snapshots). The problem only appears in the workload: `partition_prune_rate` = 0.0 reveals that the current partition on `event_date` never fires for queries that filter on `tenant_id`.

The problem is that the table is partitioned by the wrong column for its actual query pattern. This is invisible to any profiler that reads only metadata.

The skill correctly identified `tenant_id` as the key column to optimise on and recommended z-order compaction. The 4/5 score reflects the judge’s finding that **partition evolution** would be a stronger primary fix: adding `tenant_id` as a bucket partition field eliminates full-table scans without any data rewrite, using Iceberg’s metadata-only `ALTER TABLE...ADD PARTITION FIELD` syntax. Z-order compaction is correct but more expensive (full data rewrite).

6.5 `snapshot_bloat_only`: Don’t Fix What Isn’t Broken

The 1 200-snapshot table tests whether the skill avoids treating snapshot bloat as a data-file problem. The data files are exactly at target size (256 MB, 5 files) — adding compaction would rewrite perfectly-sized files for no gain.

The skill correctly:

- Identified 1 200 snapshots as the sole problem signal.
- Recommended only `expire_snapshots` with a 2-week retention window.
- Explicitly stated that data file compaction is not needed.

7 Skill Review

A qualitative review of all six skill files was conducted alongside the benchmark. The skill was evaluated on frontmatter quality, phase flow completeness, decision framework correctness, interview structure, procedures accuracy, scheduling accuracy, and signal-to-noise ratio.

7.1 Strengths

- **Trigger/gate separation.** The decision framework clearly distinguishes metadata triggers (“small files exist”) from intent gates (“queries frequent enough to amortise the rewrite”).

This prevents the classic over-engineering failure mode.

- **“Do nothing” as a first-class outcome.** Action Z is documented with the same level of detail as aggressive compaction. The simulator’s Do-nothing baseline makes this concrete.
- **Compliance path.** The GDPR sequence is correct, ordered, and explicitly overrides the query-frequency gate — the most common mistake in GDPR + Iceberg guidance is treating logical deletion as sufficient.
- **Derive-first interview.** Phase 2a derives all answerable signals from metadata before asking the user anything. This reduces interview fatigue and prevents the skill from asking questions it already knows.
- **Honest about heuristics.** `scan_fraction` (the post-optimisation improvement multiplier) is explicitly called a heuristic in SKILL.md, with instructions for how to replace it with measured numbers.

7.2 Top 3 Recommended Improvements

1. **[High impact] Elevate partition evolution for zero-prune scenarios (decision-framework.md, Step 2 Action D).**

Currently, Sort/Z-order (B/C) ranks above Partition evolution (D) in the ROI table. This is correct in the general case but inverts for tables where `partition_prune_rate = 0.0` and the top filter column is not in the partition spec. In that case, partition evolution is a metadata-only operation that eliminates full-table scans permanently, while z-order compaction is a full data rewrite that only improves data skipping.

Recommended addition to Action D gate:

If `partition_prune_rate < 0.2` AND the dominant filter column is not in the current partition spec, promote D above B/C in ROI ranking.

Also add to the worked examples:

Hot events table, partitioned by `event_date`, queried 98% by `tenant_id`: D partition evolution (add `bucket(N, tenant_id)` as leading partition field) → optional B sort within partitions → F snapshot expiry. Z-order is a less efficient alternative that requires a full data rewrite.

2. **[Medium impact] Close the late-data loop to actionable gates (workload-interview.md Part 1 + decision-framework.md).**

The skill correctly derives late-data presence (`late_data = true`) from snapshot summaries, but does not connect this signal to a specific decision gate. A table with late-arriving data breaks the “compact only cold partitions” assumption: late writes land in partitions that appear cold, causing conflicts between the compaction job and live writers.

Recommended additions:

- In decision-framework.md Action B gate: append “AND `late_data ≠ true` (late data invalidates time-column sort ordering)”.
 - In scheduling.md, streaming section: “If `late_data = true`, widen the cold-partition window by the observed maximum late-arrival lag; use threshold triggers rather than a fixed cron schedule”.
3. **[Medium impact] Provide a “no query logs” fallback in SKILL.md Phase 2 (SKILL.md, Phase 2a).**

SKILL.md describes two ways to obtain measured bytes-scanned (Spark event log, Trino system queries) but does not have a documented fallback for when neither is available. The current text refers the user to `--explain-analyze` in passing, but does not state explicitly:

If no query logs are available, ask the user to run `EXPLAIN ANALYZE` on 3–5 representative queries and pass the output via `--explain-analyze`. If even that is unavailable, the simulator will estimate from $total_gb \times (1 - prune_rate)$ — clearly label this as a heuristic in the output.

This makes the “cold-start” path explicit and sets user expectations about estimation quality.

7.3 Overall Skill Rating

Table 9: Skill quality ratings by dimension

Dimension	Rating	Notes
Frontmatter / description	9/10	Precise trigger keywords; could add “partition evolution” and “d
Phase flow completeness	7/10	No-query-log fallback missing; edge cases under-doc
Decision framework correctness	8/10	Partition evolution under-ranked for zero-prune
Interview structure	8/10	Derive-first is excellent; late-data loop not clo
Procedures accuracy	9/10	Syntax correct; Glue prerequisites slightly under-ex
Scheduling accuracy	8/10	Matrix is solid; late-data / threshold coordination
Signal-to-noise	7/10	metadata-tables.md is thorough but unprioriti
Overall	7.8/10	Benchmark confirms competence; three actionable im

8 Benchmark Infrastructure

8.1 File Structure

```
skills/iceberg-optimizer/
  tests/
    skill_benchmark/
      run_benchmark.py      # Main harness (claude CLI, retry, judge)
      scenarios.json        # 5 scenarios with expected_outcome + assertions
      generate_fixtures.py  # Deterministic fixture generator
      fixtures/
        cold_archive/      # profile.json, workload.json, simulate_output.txt
        streaming_thin_spread/
        gdpr_deletes/
        partition_misalignment/
        snapshot_bloat_only/
```

8.2 Usage

```
cd skills/iceberg-optimizer

# Run one scenario with full LLM judge output
python tests/skill_benchmark/run_benchmark.py \
  --scenario gdpr_deletes --judge --verbose

# Run all scenarios with judge, save results
python tests/skill_benchmark/run_benchmark.py \
  --all --judge --output-json results.json
```

```
# Regenerate fixtures (e.g. after changing simulate.py assumptions)
python tests/skill_benchmark/generate_fixtures.py
```

8.3 Lessons Learned

Nested CLI calls are slow. Running `claude -p` from within a Claude Code session is subject to rate limiting and session overhead. Each call takes 2–5 minutes for a complex scenario. The harness uses a 600-second timeout with 3-retry exponential backoff and a 3-second pause between scenarios.

Keyword matching fails on “what we’re NOT doing” sections. A good skill explicitly lists skipped actions with reasons. Forbidden keyword checks trip on these sections, producing false negatives. LLM judge evaluation is strictly more robust.

expected_outcome beats rubric scoring. The judge performs better when given a concrete expected outcome (“the correct answer is X because Y”) than an abstract rubric (“score 1–5 for correctness”). The `expected_outcome` field in `scenarios.json` serves this role.

9 Conclusion

The iceberg-optimizer skill passes all five benchmark scenarios with an average LLM judge score of 4.8/5. Key design decisions validated by the benchmark:

1. **The “do nothing” path works.** The skill correctly chose minimal action for the cold archive, including an explicit explanation of why maintenance would be net-negative.
2. **The GDPR compliance path is correct.** The non-optional compaction + snapshot-expiry sequence was produced without prompting.
3. **The skill reads profile *and* workload together.** The partition-misalignment scenario has a healthy profile; the problem is only visible in the workload data. The skill identified it correctly.
4. **LLM judge is the right evaluation primitive.** Keyword matching produced 40% false negatives; LLM judge 0%.

The single 4/5 score (`partition_misalignment`) points to one concrete improvement: `partition_evolution` should be elevated above `z-order` in the ROI ranking when `partition_prune_rate = 0.0`. Implementing this change would likely push all five scenarios to 5/5.